Travaux Dirigés d'analyse syntaxique n°7

Licence d'informatique

———

Interface Flex-Bison, construction d'arbres abstraits

Le but de ce TD est :

— d'analyser du code source qui ressemble à du C en utilisant `bison` et `flex` ; en particulier, on transmet des informations de `flex` à `bison` par l'intermédiaire de la variable `yylval`

— de s'entrainer à construire des arbres abstraits.

▸ Exercice 1. *Coupler Flex et Bison*

1. *Faites une nouvelle version du programme* `flex` *de votre projet pour produire un analyseur lexical qu'on puisse coupler avec* `exp-tpc.y`*. L'analyseur lexical doit aussi accepter les commentaires entre* `/*` *et* `*/` *et après* `//`*.*

   Pour vos tests, le fichier `exp.tpc` *contient une expression conforme à la grammaire de* `exp-tpc.y`*.*

2. *Modifiez* `exp-tpc.y` *et votre programme* `flex` *pour que l'analyseur syntaxique affiche des traces indiquant les opérateurs* `+` *et* `-` *(binaires comme dans* `a+b` *ou unaires comme dans* `a=-b`*),* `*`*,* `/`*,* `\%`*,* `==`*,* `!=`*,* `<`*,* `<=`*,* `>`*,* `>=` *et les appels de fonctions, avec le nom de la fonction. Par exemple, quand on analyse l'expression* `distance(value((-b)/(2*a)))`*, on veut voir apparaitre :*

```
unary -
*
/
fn value
fn distance
```

   Pour transmettre les informations de l'analyseur lexical à l'analyseur syntaxique, utilisez la variable `yylval` *définie automatiquement par* `bison`*.*

▸ Exercice 2. *Construire des arbres abstraits*

(adapté de l'examen de deuxième session du 3 juillet 2023)

On donne un programme `bison` *qui analyse un flux de caractères comme* `{ 1 2 3 4 }` *et le traduit en arbre abstrait :*

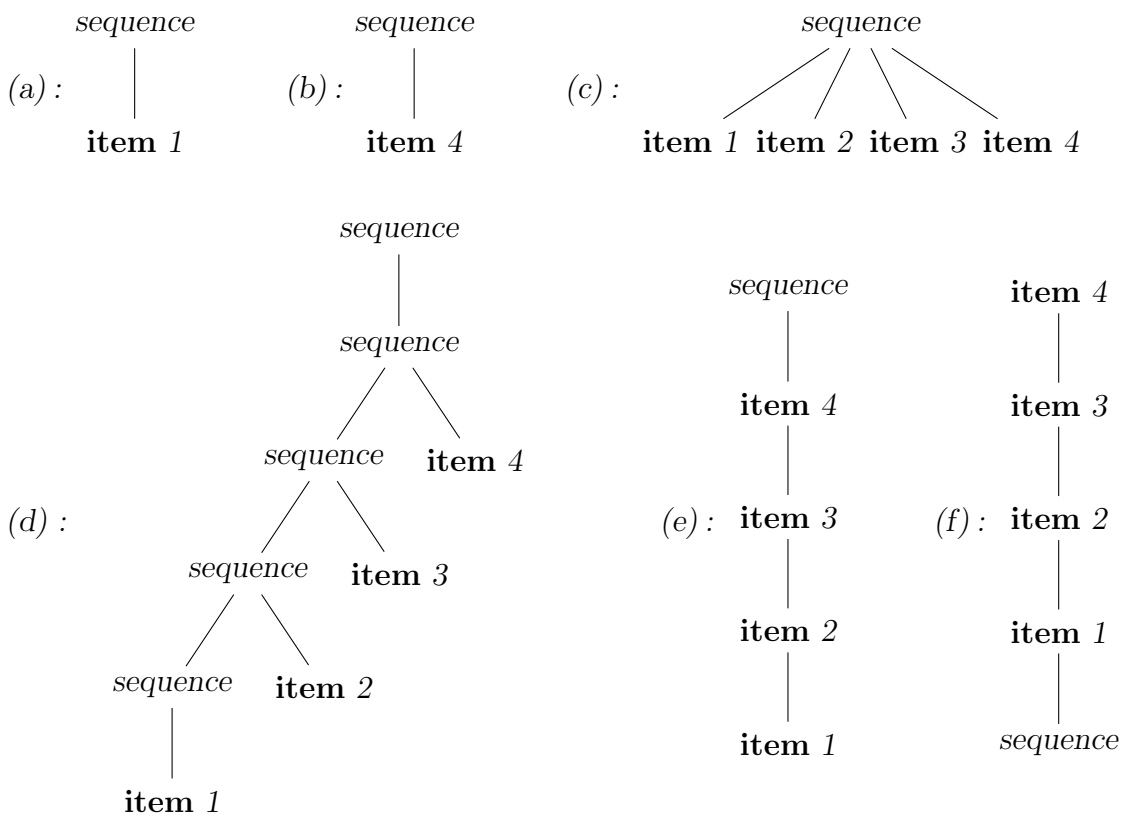
```
%{
/* list2tree.y */
/* Traduction en arbre */
#include <stdio.h>
#include "tree.h"
int yylex();
void yyerror(char *);
%}
%union {
    int num;
    Node *node;
}
%token <num> ITEM
%type <node> SequenceItems
%%
List         :  '{' SequenceItems '}' { printTree($2); }
             ;
SequenceItems: SequenceItems ITEM     { Node *i;
                                        $$=$1;
                                        i=makeNode(item);
                                        i->attribute=$2;
                                        addChild($$, i); }
             |                        { $$=makeNode(sequence); }
             ;
%%
int main(int argc, char** argv) {
  return yyparse();
}
```

On compile ce programme avec le programme `flex` *ci-dessous et avec un module* `tree.c`.

```
%{
/* items.lex */
#include "tree.h"
#include "list2tree.h"
int lineno=1;
%}
%option noinput nounput noyywrap
%%
[ \t\r]+     ;
\n           lineno++;
[0-9]+       { yylval.num=atoi(yytext); return ITEM; }
.            { return yytext[0];}
%%
```

Le module `tree.c` *et le fichier* `tree.h` *définissent* `Node`, `makeNode()`, `addChild()`, `addSibling()`, `printTree()` *et* `FIRSTCHILD()` *comme dans le projet d'analyse syntaxique.*

1. *Parmi les 6 arbres ci-dessous, quel est celui affiché par ce programme quand on lui passe le flux d'entrée* `{ 1 2 3 4 }` *?*



2. *Lequel des 6 arbres est affiché si on donne aux règles* `SequenceItems` → `SequenceItems` **ITEM** *et* `SequenceItems` → ε *les actions suivantes ?*

(ii)
```
SequenceItems: SequenceItems ITEM  { $$=makeNode(item);
                                     $$->attribute=$2;
                                     addChild($$, $1); }
             |                     { $$=makeNode(sequence); }
             ;
```

(iii)
```
SequenceItems: SequenceItems ITEM  { $$=$1;
                                     if (FIRSTCHILD($$)==NULL) {
                                        Node *i=makeNode(item);
                                        i->attribute=$2;
                                        addChild($$, i); } }
             |                     { $$=makeNode(sequence); }
             ;
```

(iv)
```
SequenceItems: SequenceItems ITEM  { Node *i;
                                     $$=makeNode(sequence);
                                     i=makeNode(item);
                                     addChild($$, i);
                                     i->attribute=$2; }
             |                     { $$=makeNode(sequence); }
             ;
```

(v)
```
SequenceItems: SequenceItems ITEM  { Node *i;
                                     $$=$1;
                                     i=makeNode(item);
                                     i->attribute=$2;
```

```
                                        addChild(i, FIRSTCHILD($$));
                                        FIRSTCHILD($$)=i; }
                  |                   { $$=makeNode(sequence); }
                  ;
```

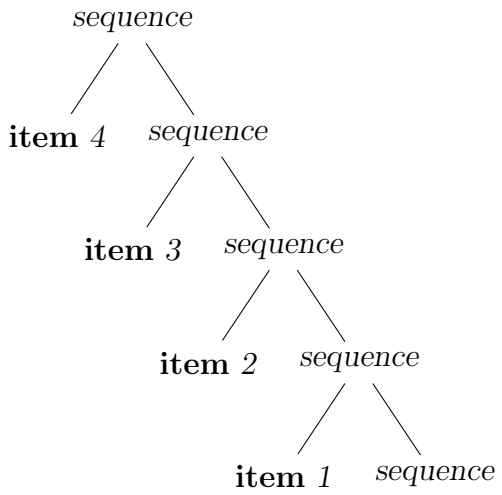
(vi)
```
SequenceItems: SequenceItems ITEM  { Node *i;
                                     $$=makeNode(sequence);
                                     i=makeNode(item);
                                     i->attribute=$2;
                                     addChild($$, $1);
                                     addChild($1, i); }
                  |                  { $$=makeNode(sequence); }
                  ;
```

Indication : chacun des 6 arbres est affiché par un des 6 programmes des questions 2.1 et 2.2. Remarque : certains de ces programmes provoquent une fuite de mémoire.

3. *Lesquels des programmes des questions 2.1 et 2.2 provoquent une fuite de mémoire ?*
4. *Dessinez l'arbre obtenu avec les règles suivantes :*

```
SequenceItems: SequenceItems ITEM  { Node *i;
                                     $$=makeNode(sequence);
                                     i=makeNode(item);
                                     i->attribute=$2;
                                     addChild($$, i);
                                     addChild($$, FIRSTCHILD($1)); }
                  |                 { $$=makeNode(sequence); }
                  ;
```

5. *Quelles actions faut-il donner aux règles* `SequenceItems` $\rightarrow$ `SequenceItems` **ITEM** *et* `SequenceItems` $\rightarrow \varepsilon$ *pour avoir l'arbre ci-dessous ?*



▸ **Exercice 3.** ***Déroulement d'une analyse LR***

On considère la grammaire suivante et son tableau d'analyse LR :

$$S \rightarrow S\ S\ + \mid S\ S\ * \mid \texttt{id}$$

État	$+$	$*$	`id`	\$	S
0			$e1$		2
1	$r3$	$r3$	$r3$	$r3$	
2			$e1$	acc	4
3					
4	$e5$	$e6$	$e1$		4
5	$r1$	$r1$	$r1$	$r1$	
6	$r2$	$r2$	$r2$	$r2$	

Analyser pas à pas la chaîne `id id * id + $`.